

Flutter

- Framework para criação de UI.
- Código em linguagem Dart.
- Produz código de máquina para Android e iOS.

Dart

<https://dart.dev>

Teste online: <https://dartpad.dev>

Android e iOS

- Android: pode ser criado onde instalar o Android Studio.
- iOS: somente em máquinas com macOS.

Configuração Flutter

Instalação em <https://flutter.dev> **Get started:**

- Flutter SDK
- Git
- Android Studio
- Visual Studio Code
- Extensões (Flutter e Dart no VS Code)

Teste

Vamos usar Windows para testar os projetos.

Primeiro projeto

1. Abrir VS Code
2. View->Command palette :
3. Digite: Flutter: new project
4. Selecione projeto vazio **Empty**
5. Selecione uma pasta
6. Dê um nome ao projeto
7. Abra lib/main.dart
8. Selecione o dispositivo

`lib/main.dart`

Altere o código em `Text()` para "Flutter no IFSP"

Terminal

Abra o terminal e execute o comando:

```
flutter run
```


Apoio

<https://youtu.be/Gza-CKpoicE>

Material Design

User Interface com visual Android/Google.

<https://docs.flutter.dev/ui>

Pastas do projeto

Crie um novo projeto: primeira_app

Pastas e arquivos do projeto

- `lib` - arquivos `.dart` da aplicação.
- `android`, `macos`, `windows`, `web`, `linux` - arquivos das plataformas (Flutter).
- `build` - arquivos gerados pelo Flutter.
- `test` - código Dart para testar a app.
- `.nome` - configuração extra (Flutter)
- `.gitignore` - Ajustes do Git (Flutter)
- `.metadata` - Configurações da app (Flutter)

- `analysis_options.yaml` - Configurações avançadas, avisos.
- `first_app.iml` - Configurações da app (Flutter)
- `pubspec.yaml` - Configurações de pacotes da app (Flutter e você).
- `pubspec.lock` - Ligado ao `pubspec.yaml`.
- `README.md` - Descrição e informações da app.

Compilação

A plataforma, exemplo Android, não entende o código Dart.
Na hora de executar:

1. O código é analisado.
2. O código é traduzido (compilado) para a linguagem de máquina da plataforma.
3. O resultado é executado.

Palavras no código `main.dart`:

- keywords - criadas na linguagem Dart (import, void, class).
- identificadores - nomes de partes do código (classes, variáveis).

Funções

- `main()` - onde código Dart inicia.
- `runApp()` - definido no Flutter (pubspec.yaml, dependencies)
- `import` - importar código de outros arquivos Dart.

main.dart básico

```
import 'package:flutter/material.dart';  
  
void main() {  
  runApp(MaterialApp());  
}
```

Argumentos

```
// nenhum argumento em main
void main() {
    // um argumento sem nome em runApp
    runApp(MaterialApp());
}
```

Argumentos

```
// nenhum argumento em main
void main() {
    // um argumento sem nome em runApp
    // um argumento com nome, home, em MaterialApp
    runApp(MaterialApp(home: Text('Hello, world!')));
}
```

Prática

- Altere 'Hello, world!' para 'Olá, IFSP!'.

Argumentos

```
void main() {  
    print(soma(3, 2));  
}  
  
int soma(int x, int y) {  
    return x + y;  
}
```

Argumentos nomeados

- Chaves `{` e `}`:

```
void main() {  
    print(soma(x: 3, y: 2));  
}  
  
int soma({required int x, required int y}) {  
    return x + y;  
}
```

Deixe o mouse sobre `Text()`, veja os argumentos na documentação!

```
runApp(  
  MaterialApp(  
    home: Text('Hello, world!')));
```

Text()

O primeiro argumento é uma String, o restante dos argumentos de **Text** são nomeados.

Valores `const`

O Flutter identifica itens que podem ser ajustados com `const` para melhorar a performance da aplicação:

Use 'const' with the constructor to improve performance.

Quando aparecer, use Quick Fix!

Scaffold

Localizar o Widget:

- Docs do Flutter
- Widget catalog
- Basics

O widget [Scaffold](#) implementa o visual Material Design por nós:

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    const MaterialApp(
      home: Scaffold(
        body: Text('Hello, world!'),
      ),
    ),
  );
}
```

Center

Para deixar um elemento centralizado em Flutter usamos o widget `Center`

Para acessar:

`Widget catalog->Base widgets->Layout->Center`

Botão direito em `Text()`, `refactor`, e `wrap with Center`.

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    const MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text('Hello, world!'),
        ),
      ),
    ),
  );
}
```

Prática

Pesquise em Scaffold, como mudar a cor de fundo, altere a cor do fundo.

Prática

Opções: `Color.fromARGB()` ou `Colors.blue`:

```
home: Scaffold(  
  backgroundColor: Colors.lightBlue,  
  body: Center(  
    child: Text('Hello, world!'),  
  ),  
)
```

Container

Outra opção para cor de fundo: gradiente.

Scaffold não aceita, então vamos colocar o Center dentro de um Container e ajustar a decoration dele:


```
MaterialApp(  
  home: Scaffold(  
    //backgroundColor: Colors.lightBlue,  
    body: Container(  
      decoration: BoxDecoration(  
        gradient: LinearGradient(colors: [  
          Colors.blue,  
          Colors.lightBlue,  
        ]),  
      ),  
      child: Center(  
        child: Text('Hello, world!'),  
      ),  
    ),  
  ),  
)
```